

arineto1.github.io/ifpe2026

Organização Estruturada de Computadores

Aula 5

O Conceito de Processamento Sequencial vs. Pipeline

Como o Processador Funciona? (O Ciclo de Instruções)

O **processador (CPU)** é o **cérebro do computador**, mas ele não toma decisões sozinho. Ele é um executor de ordens extremamente obediente. Todo o **software do computador (do Windows ao WhatsApp)** é **quebrado em milhões de pequenas instruções**. A CPU funciona seguindo um ciclo infinito de quatro **passos básicos**.

A CPU funciona seguindo um **ciclo infinito de quatro passos básicos**:

Busca (Fetch - FI): A CPU vai até a memória (geralmente na memória Cache L1) e busca a próxima instrução que precisa ser executada.

Decodificação (Decode - DI): O processador analisa a instrução para entender o que ela quer fazer (Se é uma soma, uma subtração, ou se precisa buscar um dado na memória RAM).

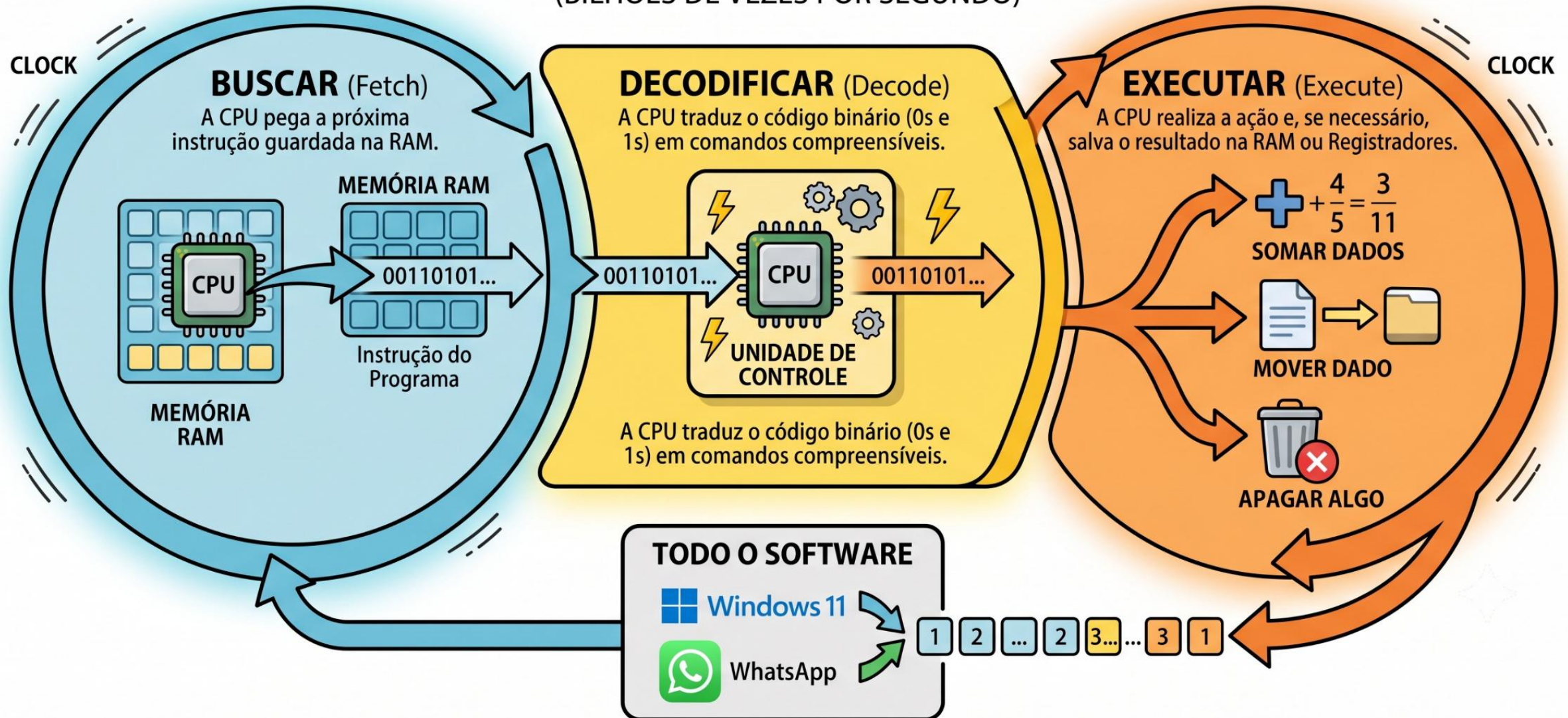
Execução (Execute - EI): A Unidade Lógica e Aritmética (ULA) entra em ação e faz o cálculo matemático ou a operação lógica real.

Escrita (Writeback - WO): O resultado final do cálculo é salvo de volta em um registrador da CPU ou na memória, finalizando o processo.

Assim que termina a etapa 4, o **processador volta para a etapa 1** e busca a próxima instrução. Esse ciclo se **repete bilhões de vezes por segundo (Clock)**.

COMO A CPU FUNCIONA: O CICLO DE INSTRUÇÃO

CICLO BUSCA-DECODIFICAÇÃO-EXECUÇÃO (BILHÕES DE VEZES POR SEGUNDO)



ANALOCIA DIDÁTICA: O Mestre-Cuca e o Livro de Receitas (CPU e RAM)

Mestre-Cuca
= CPU



Livro de
Receitas =
MEMÓRIA
RAM

O CICLO DE PROCESSAMENTO (Passo a Passo)

1. BUSCA (Fetch)



*A CPU "lê" a próxima instrução da memória.

2. DECODIFICAÇÃO (Decode)



*O "cérebro" do processador interpreta a instrução.

3. EXECUÇÃO (Execute)



*A CPU realiza a operação solicitada (fazer a ação).

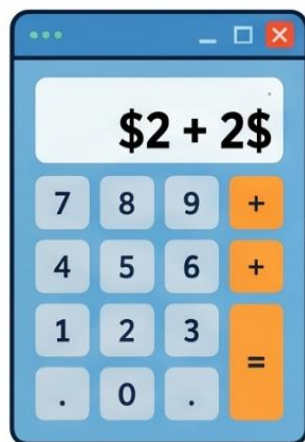
..... REPETIÇÃO DO CICLO <.....

*Finalizada a ação, o Mestre-Cuca volta **IMEDIATAMENTE** para ler a **PRÓXIMA** linha: "2. Adicione 2 xícaras de açúcar" e o processo recomeça.

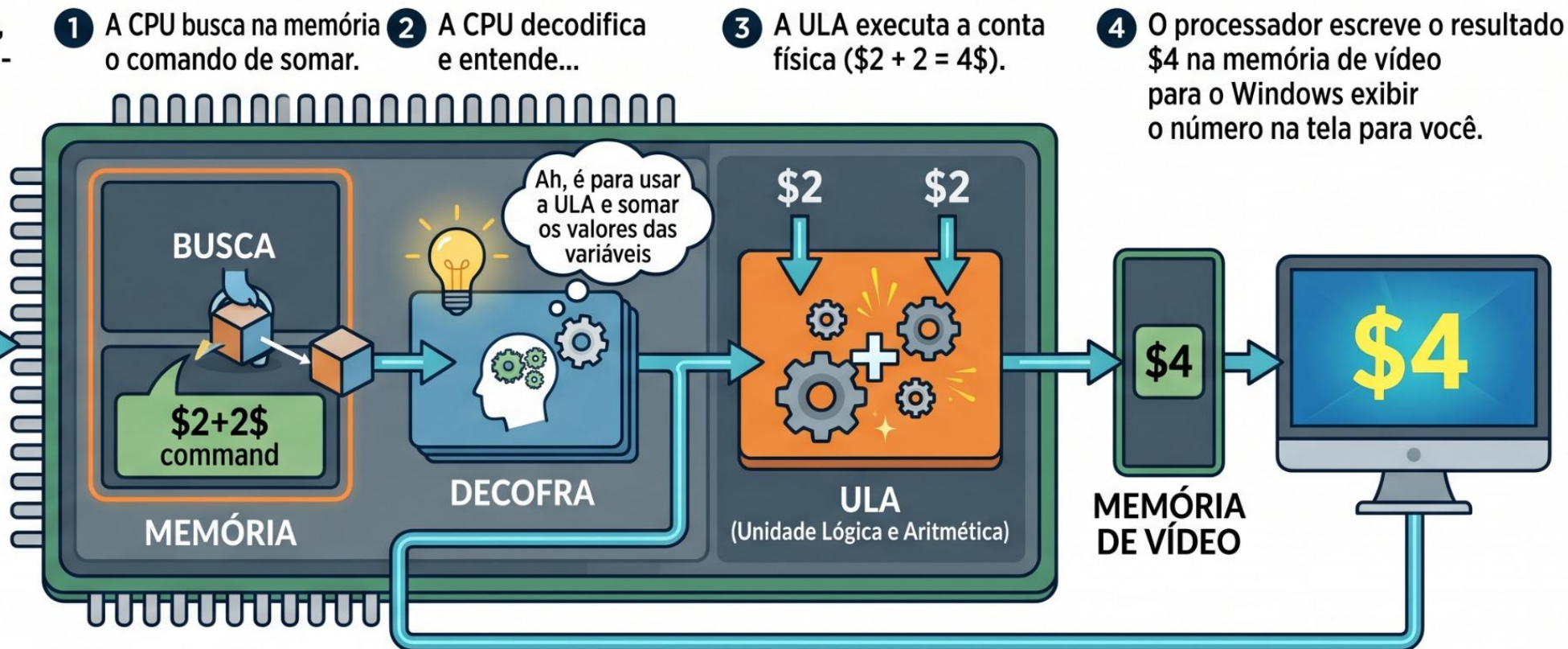
O processador passa o dia inteiro fazendo exatamente isso, linha por linha!

Exemplo Prático: Como a CPU Processa 2 + 2

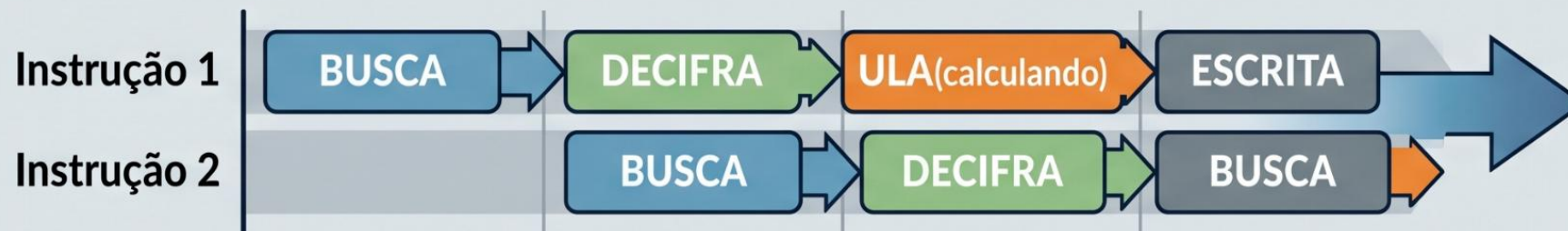
No nível mais baixo do hardware, o Windows ou o aplicativo da calculadora envia esses comandos para o processador:



Comando de somar



COM O PIPELINE:



enquanto a ULA está calculando o seu $\$2 + 2\$$, a CPU já está buscando o próximo comando do teclado que você vai digitar logo em seguida.

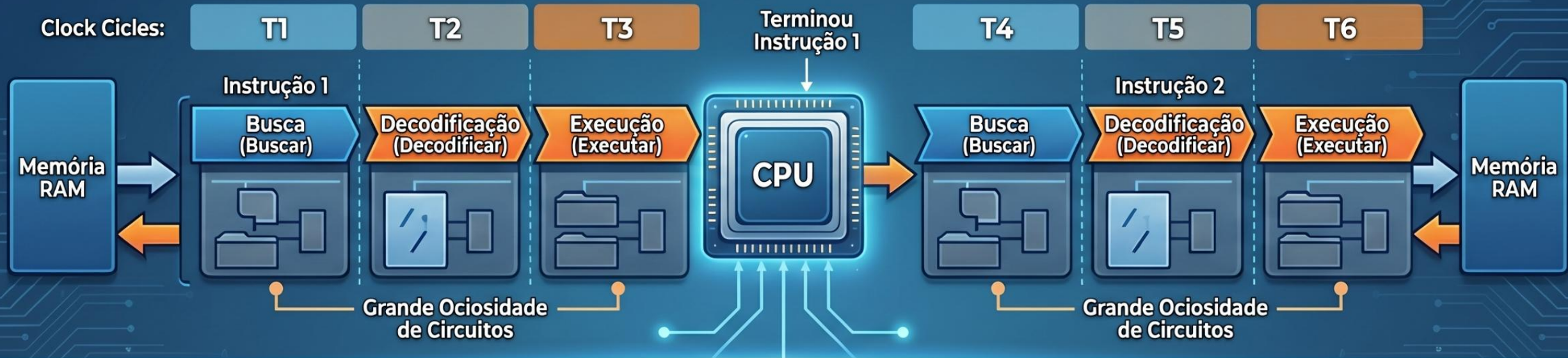
O Conceito de Pipeline

Processamento Sequencial: Antigamente, o processador executava uma instrução inteira do início ao fim (**Busca, Decodificação, Execução**) e, **somente quando ela terminava**, ele buscava a próxima instrução na memória RAM. **A maior parte dos circuitos ficava ociosa esperando.**

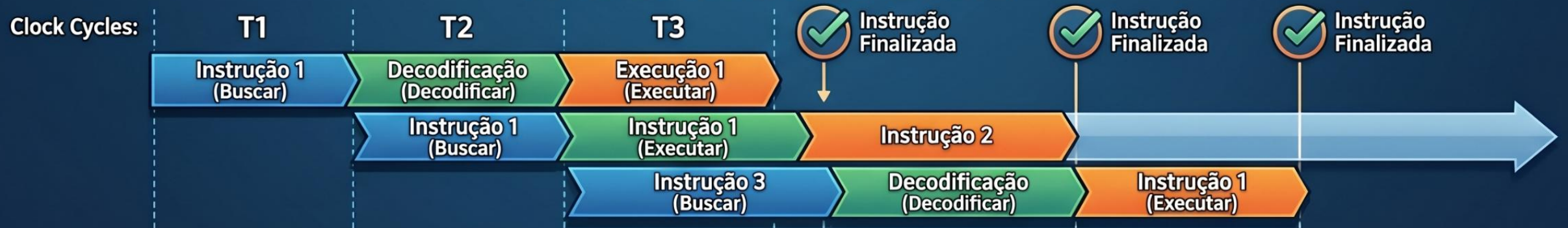
Processamento em Pipeline: É uma técnica de hardware que permite a **sobreposição de estágios de execução**. Enquanto a primeira instrução está na fase de execução, a segunda já está sendo decodificada e a terceira já está sendo buscada na memória.

O Objetivo: O Pipeline não diminui o tempo que uma única instrução leva para ser executada (latência), mas **aumenta drasticamente a quantidade de instruções finalizadas por segundo (vazão/throughput).**

PROCESSAMENTO SEQUENCIAL



PROCESSAMENTO EM PIPELINE



O OBJETIVO:



SEM ALTERAÇÃO DE LATÊNCIA

Duração de uma instrução em ambos os modelos é igual

=



AUMENTO DE VAZÃO (THROUGHPUT)

Aumento de instruções completadas por segundo, com Pipeline.

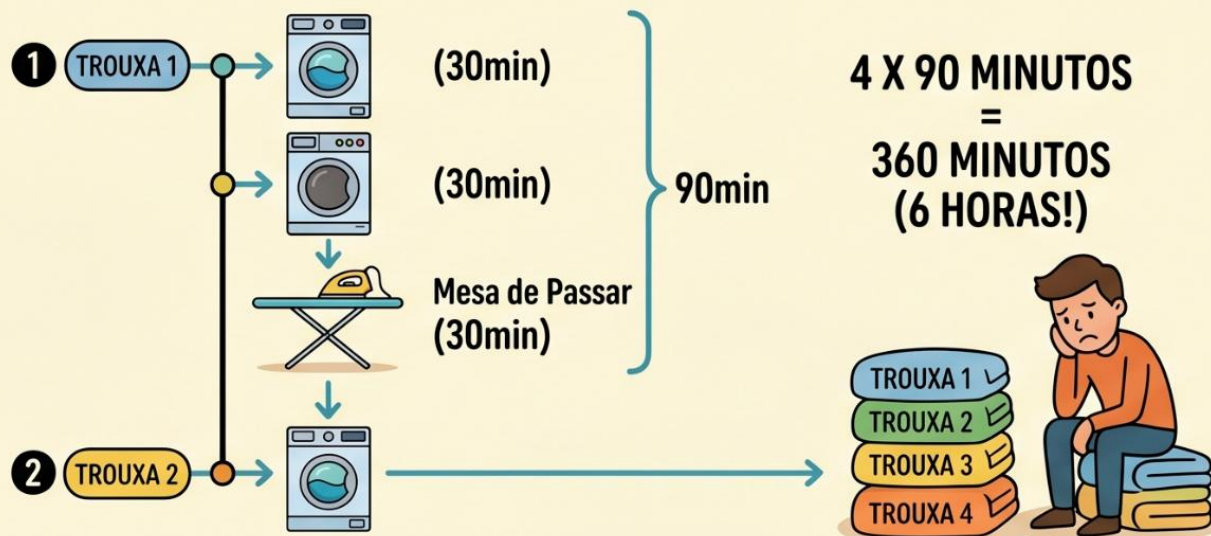


ANALOGIA DIDÁTICA: O QUE É PIPELINE?

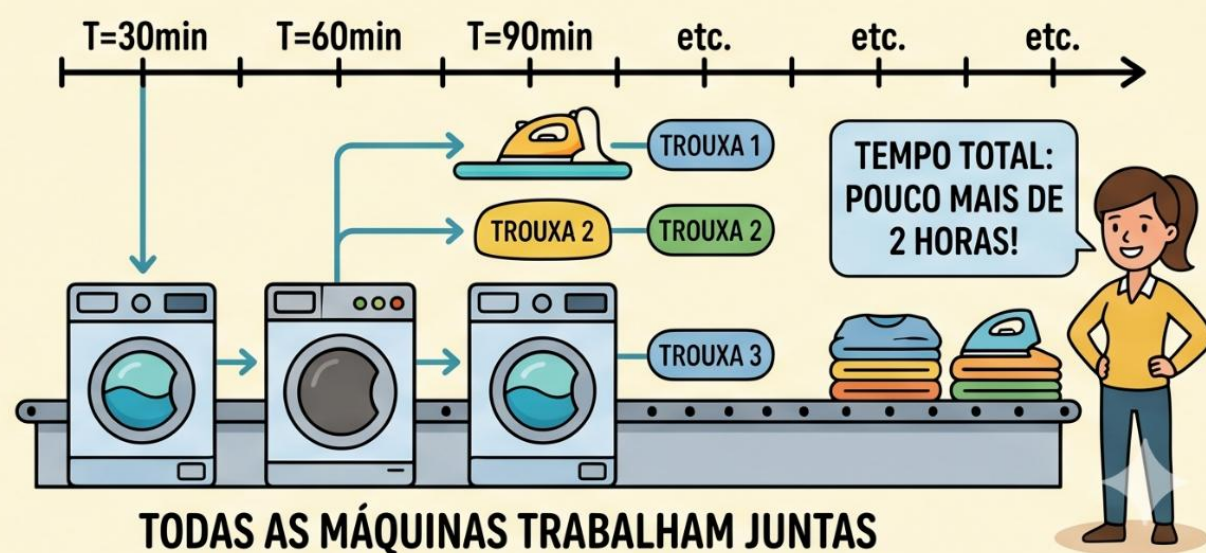
O PROBLEMA DAS 4 TROUXAS



MODO SEQUENCIAL (SEM PIPELINE)



MODO PIPELINE (COM PIPELINE)



Impacto no Desempenho e o Ganho de Velocidade (*Speedup*)

O que é o Ganho de Velocidade (*Speedup*): É a métrica que calcula quantas vezes o processador ficou mais rápido depois que ativamos o Pipeline.

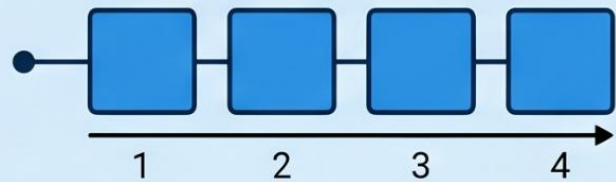
A Regra Teórica: Em um mundo perfeito, se dividirmos o nosso Pipeline em 4 estágios (como vimos no slide anterior), o computador deveria terminar as tarefas 4 vezes mais rápido.

A Realidade: Na prática, o ganho nunca é 100% perfeito porque as instruções dependem umas das outras e a memória RAM às vezes atrasa a entrega dos dados. Mesmo assim, o ganho de desempenho é gigantesco.

Atenção (O segredo do Pipeline): O Pipeline **não** faz uma instrução sozinha rodar mais rápido. O tempo de viagem de uma única instrução dentro do chip continua o mesmo. O que muda é que, como **várias instruções viajam juntas, o volume de trabalho entregue por segundo (Vazão) aumenta drasticamente.**

PROCESSADOR SEM PIPELINE (MODO SEQUENCIAL)

Tarefa 1: Estágio 1, 2, 3, 4



Uma tarefa por vez.
Total: 4 ciclos por tarefa

Tarefa 2: Estágio 1, 2, 3, 4



Tarefa 3: Estágio 1, 2, 3

PROCESSADOR COM PIPELINE (MODO IDEAL DE 4 ESTÁGIOS)

Estágio 1



Estágio 2



Estágio 3



Estágio 4

Uma tarefa completa a cada ciclo após o preenchimento. Múltiplas tarefas processadas ao mesmo tempo.



CALCULANDO O GANHO DE VELOCIDADE (SPEEDUP)

$$\text{Speedup} = \frac{\text{Tempo Total Sem Pipeline}}{\text{Tempo Total Com Pipeline}}$$

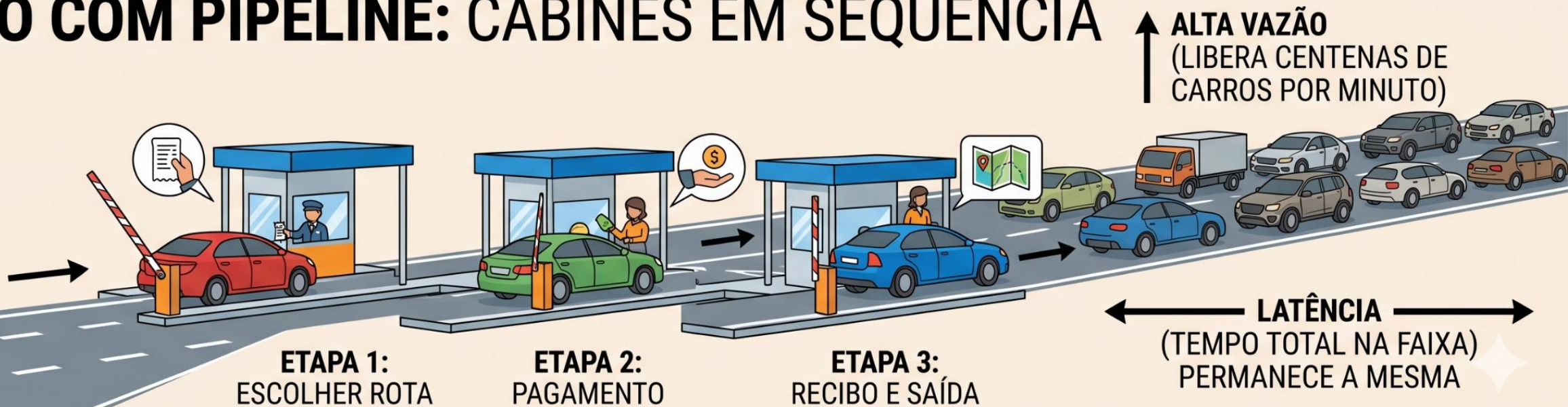
Em um mundo perfeito, se dividimos o Pipeline em 4 estágios, o computador deveria terminar as tarefas **4 VEZES** mais rápido.

RESULTADO:
Ganho de Velocidade Teórico = 4x

MODO SEM PIPELINE: UM ÚNICO GUICHÊ



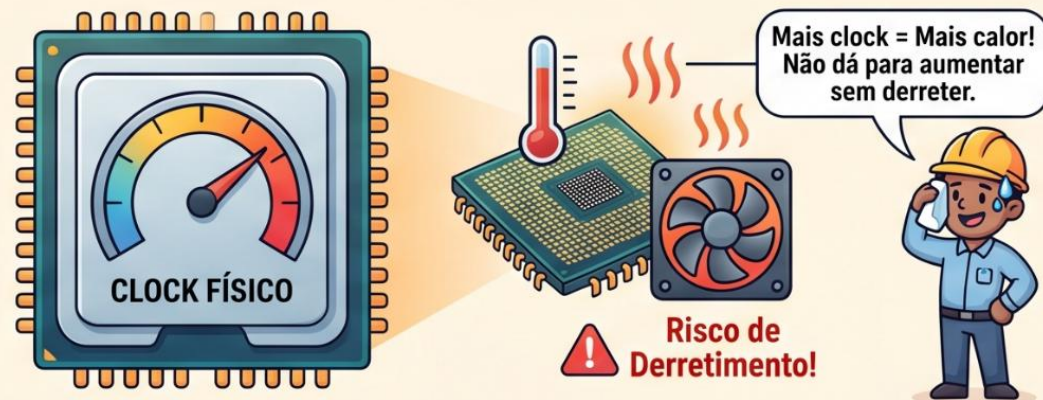
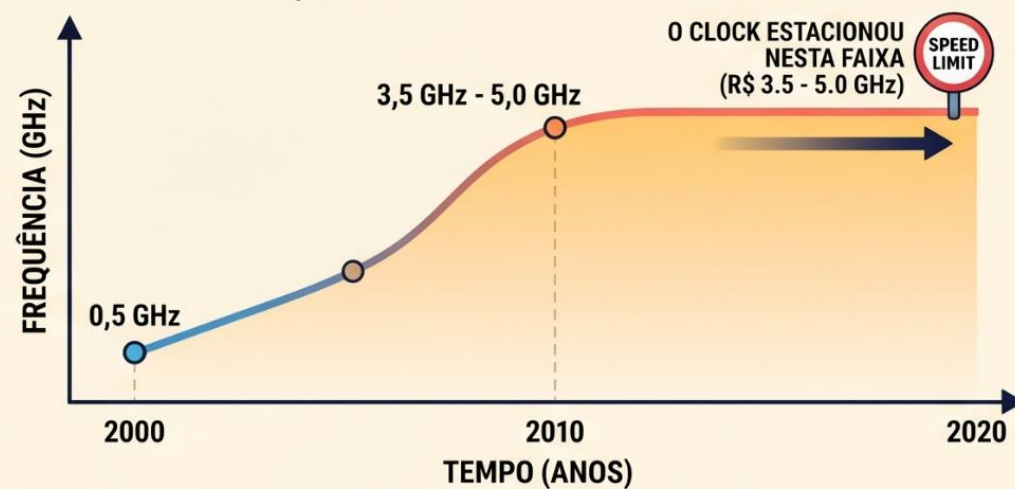
MODO COM PIPELINE: CABINES EM SEQUÊNCIA



EXEMPLO PRÁTICO: POR QUE O CLOCK PAROU E O PIPELINE É A SOLUÇÃO.

PROBLEMA: O LIMITE DO CLOCK (FREQUÊNCIA).

EVOLUÇÃO DO CLOCK DE PROCESSADORES.



SOLUÇÃO: PIPELINE PROFUNDO (DIVIDIR E CONQUISTAR)

SEM PIPELINE (UMA TAREFA POR VEZ)

INSTRUÇÃO ÚNICA: BUSCAR > DECODIFICAR > EXECUTAR > GRAVAR

INSTRUÇÃO ÚNICA: BUSCAR > DECODIFICAR > EXECUTAR > GRAVAR



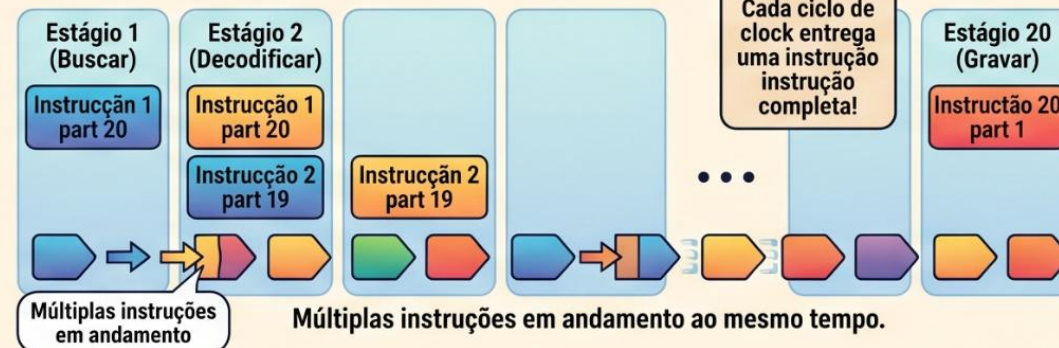
Baixo Desempenho



O "maestro" bate na mesma velocidade, mas a produção é contínua!



COM PIPELINE (20 ESTÁGIOS / LINHA DE PRODUÇÃO)



MESMO COM O CLOCK "PARADO", O PROCESSADOR ENTREGA MUITO MAIS DESEMPENHO A CADA CICLO.



Conflitos no Pipeline (*Hazards*)

Um **Hazard (ou Conflito)** ocorre quando a próxima instrução na linha de produção não pode ser executada no próximo ciclo de clock. Quando isso acontece, o Pipeline é obrigado a travar temporariamente (gerando uma pausa chamada de **Bolha** ou *Stall*), diminuindo o desempenho. Existem 3 tipos de conflitos:

Conflito Estrutural (Falta de ferramentas): Acontece quando duas instruções que estão no Pipeline precisam usar a mesma peça física do processador ao mesmo tempo. (Ex: Duas instruções tentando ler a memória RAM no mesmo milissegundo).

Conflito de Dados (Dependência): Acontece quando uma instrução precisa do resultado de uma conta que a instrução anterior ainda não terminou de calcular.

Conflito de Controle (Desvios/Decisões): Acontece quando o código tem um comando de decisão (como um IF no código). A CPU não sabe qual será a próxima instrução até que o IF seja calculado, ficando sem saber o que colocar na linha de produção.

CONFLITOS NO PIPELINE (HAZARDS): COMO O DESEMPENHO É AFETADO E OS TRÊS TIPOS

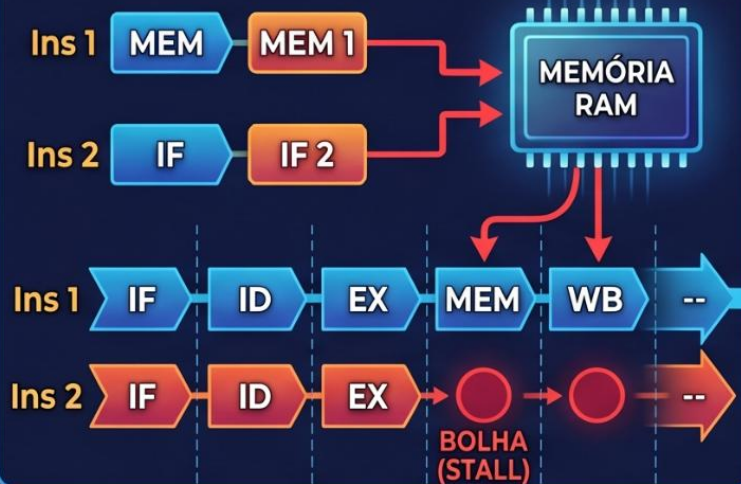
Um **Hazard** ocorre quando a próxima instrução não pode ser executada no próximo ciclo de clock. O **Pipeline trava temporariamente (Bolha/Stall)**, diminuindo o desempenho.

ESTÁGIOS DO PIPELINE (IF, ID, EX, MEM, WB)



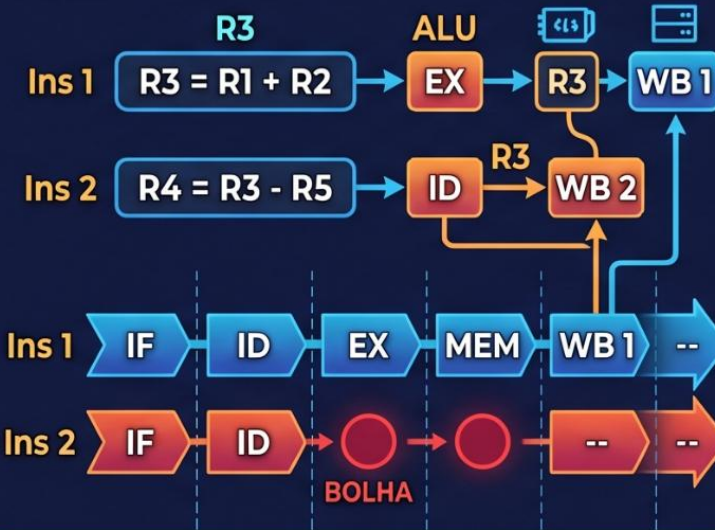
1. CONFLITO ESTRUTURAL (FALTA DE FERRAMENTAS)

Acontece quando duas instruções precisam usar a mesma peça física do processador ao mesmo tempo. (Ex: Duas instruções tentando ler a memória RAM no mesmo milissegundo).



2. CONFLITO DE DADOS (DEPENDÊNCIA)

Acontece quando uma instrução precisa do resultado de uma conta que a instrução anterior ainda não terminou de calcular.



3. CONFLITO DE CONTROLE (DESVIOS/DECISÕES)

Acontece quando o código tem um comando de decisão (como um IF). A CPU não sabe qual será a próxima instrução até que o IF seja calculado, ficando sem saber o que colocar na linha de produção.



CICLOS DE CLOCK
(t1, t2, t3...)

FLUXO COM BOLHAS
(STALLS)

FLUXO SEM BOLHAS

MENOR
DESEMPENHO

MAIOR
DESEMPENHO

Linha de Produção de Ovos de Páscoa: Tipos de Conflito

3 Conflitos na Prática



1. Conflito Estrutural
(Recurso Compartilhado)



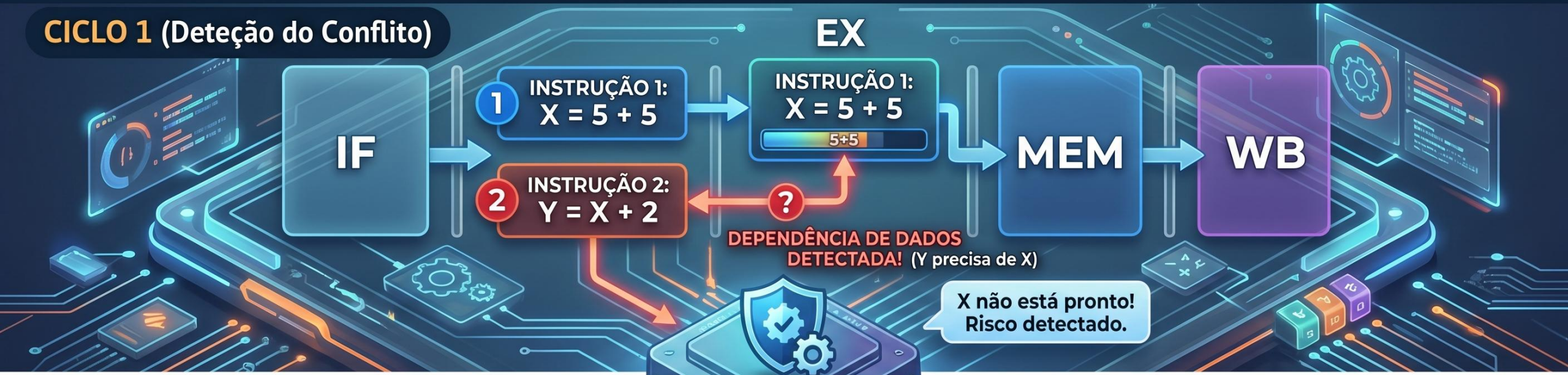
2. Conflito de Dados
(Dependência de Informação)



3. Conflito de Controle
(Incerteza na Decisão)

EXEMPLO PRÁTICO: CONFLITO DE DADOS E BOLHA NO PIPELINE

CICLO 1 (Detecção do Conflito)



CICLO 2 (Injeção de Bolha e Atraso)



Soluções para os Conflitos (Como Desengasgar o Pipeline)

Para evitar que o processador perca tempo criando "bolhas" (pausas) na linha de produção, os engenheiros criaram três soluções principais diretamente nos circuitos do chip:

Adiantamento ou Encaminhamento (*Forwarding*): Resolve os *Conflitos de Dados*. Em vez de esperar uma instrução salvar o resultado final lá na última etapa (Escrita), o circuito do processador pega o resultado "quente" direto da saída da ULA e joga imediatamente na entrada da próxima instrução.

Previsão de Desvio (*Branch Prediction*): Resolve os *Conflitos de Controle*. O processador usa um algoritmo de inteligência artificial embutido no hardware para tentar adivinhar se um comando de decisão (como um IF) vai dar verdadeiro ou falso. Ele chuta uma resposta e continua enchendo o Pipeline com o caminho que ele acha que é o certo.

Execução Fora de Ordem (*Out-of-Order Execution*): O processador analisa o código e, se perceber que uma instrução vai ficar travada esperando um dado, ele "olha para a frente" no programa, acha outra instrução independente e a executa antes, aproveitando o tempo ocioso.

SOLUÇÕES PARA EVITAR BOLHAS (PAUSAS) NA LINHA DE PRODUÇÃO DO PROCESSADOR

ENGENHARIA NO CIRCUITO DO CHIP

1 ADIANTAMENTO OU ENCAMINHAMENTO (FORWARDING)

Resolve Conflitos de Dados.



Em vez de esperar o resultado na última etapa (Escrita), o circuito **pega o resultado direto da ULA** e joga na próxima instrução.

2 PREVISÃO DE DESVIO (BRANCH PREDICTION)

Resolve Conflitos de Controle.



3 EXECUÇÃO FORA DE ORDEM (OUT-OF-ORDER EXECUTION)



Se uma instrução travar, o processador "**olha para a frente**", acha outra **independente** e a executa antes, aproveitando o tempo ocioso.

Atalho: Adiantamento



Palpite do Dono:

Previsão de Desvio



Mudar a Ordem do Trabalho:

Fora de Ordem



ILUSTRE: EXEMPLO PRÁTICO - O QUE ACONTECE QUANDO O PROCESSADOR ERRA A PREVISÃO (A QUEDA DE DESEMPENHO)

CENÁRIO 1: ALTO DESEMPENHO (PREVISÃO CORRETA)



O Jogo segue um padrão esperado. O processador acerta as decisões de desvio (branches) e o pipeline processa instruções continuamente

RESUMO DA RECUPERAÇÃO:
O *Pipeline* falha e se recupera após a limpeza, fechando a engrenagem lógica de como o processador funciona!

CENÁRIO 2: QUEDA DE DESEMPENHO (ERRO DE PREVISÃO)



Quando você está jogando e o jogo toma uma decisão imprevisível, o processador erra o palpite e precisa buscar as instruções do caminho certo do zero. É por isso que você vê uma micro-travada em um jogo

Desafio: O Circuito do $2 + 2$

O que fazer:

Pesquise e crie um **esquema visual** (desenho ou fluxograma) que explique o caminho e o funcionamento do processador ao calcular a conta $2 + 2 = 4$.

O diferencial:

Crie e ilustre uma **analogia do cotidiano** de sua preferência (ex: uma oficina, uma cozinha, uma fábrica) para explicar como os componentes internos do chip cooperam para resolver essa conta.

Entrega: Imagem do esquema + explicação da sua analogia.